

UML & Design Rationale — HW3 Skid-Steer Robot Simulation

This document models the provided starter **and** the pieces I add, and records the design decisions behind the additions. Diagrams are Mermaid (they render on GitHub).

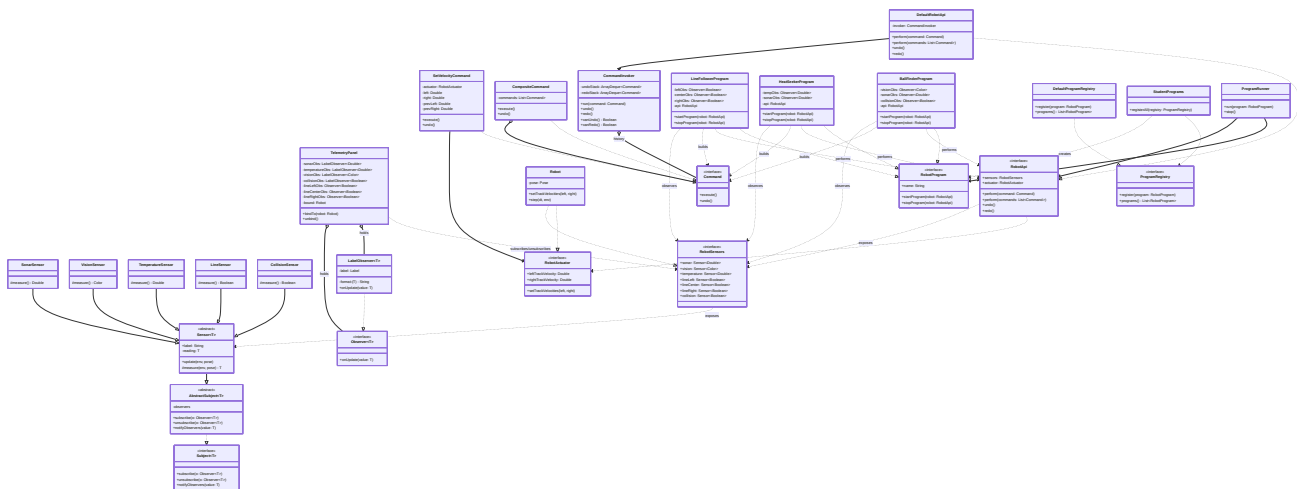
Notation

- Visibility: **+** public, **#** protected, **-** private. All stored attributes are private.
- Status tags used in the prose: **[P]** provided & working · **[P*]** provided, I fill the method body · **[S]** I author the class.

Diagram 1 — Master class diagram (Observer + Command)

One diagram for both structural patterns, because they share their key types (`Observer` , `Sensor` , `RobotSensors` , `RobotApi` , and the programs). Read it in three bands:

- **Observer** (top) — each `Sensor` is a `Subject` ; `TelemetryPanel` and the programs are the observers. `Robot` implements `RobotSensors` , so its sensors *are* the subjects.
- **Command** (middle) — `SetVelocityCommand` / `CompositeCommand` run through `CommandInvoker` behind the `RobotApi` facade; `Robot` is the `RobotActuator` receiver.
- **Programs** (bottom) — the seam: each `RobotProgram` *observes* sensors (holds `Observer` fields) and *acts* by performing commands through `RobotApi` .

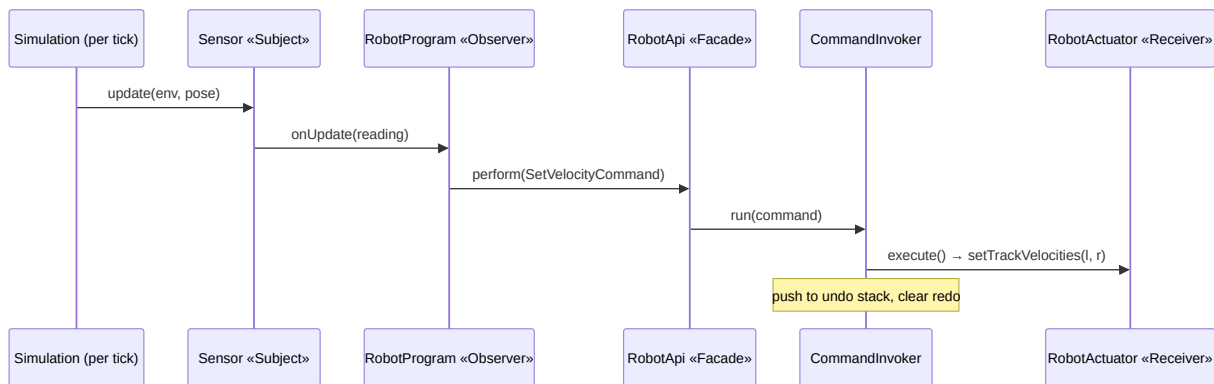


Legend. **[P]** provided & working · **[P*]** provided, I fill the method body · **[S]** I author it.

- **[P*]** `AbstractSubject` (three bodies), `CommandInvoker` (`run` / `undo` / `redo`), `StudentPrograms.registerAll` , and the bodies of `TelemetryPanel.bindTo` / `unbind` .
- **[S]** `LabelObserver` , `SetVelocityCommand` , `CompositeCommand` , and the three programs.
- **[P]** everything else — `Observer` , `Subject` , `Sensor` + subclasses, `RobotSensors` , `Command` , `RobotActuator` , `Robot` (implements both `RobotActuator` **and** `RobotSensors`), `RobotApi` , `DefaultRobotApi` , `RobotProgram` , `ProgramRegistry` , `DefaultProgramRegistry` , `ProgramRunner` .

Diagram 2 — The integration seam (where Observer meets Command)

This is the closed loop the assignment cares about: a sensor notification (Observer) becomes a `perform` call (Command), on every tick.



Constraints I do *not* get to decide (fixed by the starter)

These shape every decision below, so they are listed once here rather than re-argued each time.

- `Observer<T>` / `Subject<T>` are **generic per reading type** — `Sensor<Double>` , `Sensor<Color>` , `Sensor<Boolean>` . One object cannot implement two different `Observer<T>` .
- Sensors call `notifyObservers` **every tick**, not only on change — a subscription is a clock.
- The command **receiver is `RobotActuator`** (two track velocities), not the whole `Robot` .
- `CommandInvoker` already owns two `ArrayDeque<Command>` ; I implement the three method bodies.
- `RobotApi` is a **provided Facade** over the invoker; manual UI and programs are both its clients.

Design decisions

Each decision lists the options with pros/cons and marks the recommendation with .

D1 — Shape of the command set

Option	Pros	Cons
 Single <code>SetVelocityCommand(actuator , left, right)</code> (+ Composite, see D2)	Matches the starter's own hint; all five buttons and any program velocity flow through one class; snapshot/undo logic written once; least duplication	On its own reads as one class, not a "set" (D2 fixes this); action intent (turn vs. forward) is not encoded in the type
Abstract <code>RobotCommand</code> base + named subclasses (<code>Forward</code> , <code>Back</code> , <code>TurnLeft</code> , <code>TurnRight</code> , <code>Stop</code>)	Named intent per action; a clear "set of classes"; base centralizes undo via Template Method (an extra pattern for Req 3.4)	Five classes differing only by constants; the base pays off only with several subclasses; fixed speeds are less flexible for programs needing arbitrary velocities
Single class, no base, no composite	Absolute minimum code	Singular "class" vs. the required "set"; forgoes easy Composite credit

Recommendation: `SetVelocityCommand` as the workhorse; "Stop" is just `SetVelocityCommand( actuator , 0.0 , 0.0 )`. Choose the abstract-base route only if you specifically want named action types.

D2 — Include `CompositeCommand` ?

Option	Pros	Cons
 Include	Makes "a set of well-designed Command classes" true <i>without</i> duplicate classes; is a real pattern (Composite)	A little more code; <code>undo()</code>

Option	Pros	Cons
it	for Req 3.4; pairs with the provided <code>perform(List<Command>)</code> ; lets a program treat "turn then advance" as one undoable unit; explicitly listed under the assignment's extensions	must reverse the child order (easy to get wrong, so test it)
Omit it	Less code	Leans on the D1 subclass route for plurality; skips a clean, cheap pattern

D3 — Where the undo state lives

Option	Pros	Cons
 In the command — each captures <code>prevLeft / prevRight</code> in <code>execute()</code>	Keeps <code>CommandInvoker</code> trivial and testable with a fake <code>Command</code> (no robot needed); command is self-contained; matches the assignment's wording	Each command carries a little state
In the invoker — snapshot robot state around each <code>run</code>	Commands become stateless	Couples the invoker to the receiver's shape; breaks the moment a command changes more than velocity; harder to unit-test

D4 — Telemetry: named observer vs. inline lambdas

Option	Pros	Cons
 <code>LabelObserver<T></code> class	Reusable across all six labels; gives graders a clear <i>second</i> concrete Observer beyond the program; models cleanly in UML	One extra small class; needs a formatter (a <code>Color</code> reads differently than a <code>Double</code>)

Option	Pros	Cons
Inline <code>subscribe { v -> label.text = ... }</code> lambdas in <code>bindTo</code>	Least code; idiomatic for a Kotlin <code>fun interface</code>	No named Observer type in the diagram; formatting logic scattered across <code>bindTo</code> ; an anonymous subscription cannot be unsubscribed

Every telemetry observer is stored as a field — the four single-value labels use `LabelObserver`, and the three line sensors (which share one L/C/R label) use named `Observer<Boolean>` fields rather than inline lambdas. Storing all of them is what lets `unbind` / `bindTo` remove them again (see D5).

D5 — How a program observes (has-a vs. is-a)

Option	Pros	Cons
✓ Program <i>has</i> observers — stores each as a field, subscribes in <code>startProgram</code> , unsubscribes in <code>stopProgram</code>	Works with the generic per-type sensors; composition over inheritance; clean teardown; caches readings from multiple sensors	Must retain each observer reference to unsubscribe; the program coordinates several callbacks
Program <i>is</i> an Observer — implements <code>Observer<T></code> directly	One fewer object when a single sensor is used	Impossible when subscribing to sensors of different <code>T</code> (type erasure) — e.g. the ball finder needs <code>Color</code> and <code>Double</code> ; not viable in general

Correctness note carried by this decision: **both the telemetry panel and the programs store every observer as a field and unsubscribe it. A program must `unsubscribe` in `stopProgram`**, because it stops while the same robot lives on — otherwise it leaks observers and keeps driving. The **telemetry panel** is re-bound to a fresh `Robot` on every reset/environment change; even though the old subject is discarded, `bindTo` first calls `unbind()` to detach from the previous robot (and `unbind()` is available for disposal), so subscriptions never accumulate. This requires that the panel keep named references to *all* its observers — including the three line-sensor observers — so nothing is left as an un-removable anonymous subscription.

D6 — One program per environment

Option	Pros	Cons
✓ Three programs (<code>LineFollowerProgram</code> , <code>HeatSeekerProgram</code> , <code>BallFinderProgram</code>), each registered	Each is an interchangeable Strategy selected from the dropdown (Req 3.4); self- contained; one per objective; small enough to test	Three classes to write and verify
One program that branches on the environment	Fewer classes	Violates SRP/OCP; not Strategy; a growing <code>if/when</code> that is hard to reason about

D7 — Depend on interfaces, not concretes (DIP)

Programs reference only `RobotApi` , `Command` , and `Observer` — never `Robot` or `DefaultRobotApi` . This is Req 3.4 made structural, and it is why the diagram shows a program's only outward edges going to interfaces. Pro: swappable, testable, honors the facade. Con: none worth noting here.

Patterns present (for Req 3.4)

Pattern	Where	Provided or mine
Observer	<code>Subject</code> / <code>Observer</code> / <code>AbstractSubject</code> ; sensors → telemetry & programs	pattern mine to implement
Command	<code>Command</code> , <code>SetVelocityCommand</code> , <code>CommandInvoker</code>	mine
Composite	<code>CompositeCommand</code>	mine
Strategy	interchangeable <code>RobotProgram</code> s in the dropdown	mine
Template Method	only if D1's abstract-base route is chosen	mine (optional)
Facade	<code>RobotApi</code> / <code>DefaultRobotApi</code> over the invoker	provided

Pattern	Where	Provided or mine
Registry / Simple Factory	<code>ProgramRegistry</code> + <code>StudentPrograms.registerAll</code>	provided hook, I register